# Code Review: agent-sh/computer-use-linux

**Strong candidate** · Rust · MIT license · v0.7.12

This project could be a useful addition to ContextBridge, particularly for controlling Friday's Linux desktop. It covers a major capability that Desktop Commander's traditional filesystem and terminal tools do not address directly: graphical desktop interaction.

The project is a native Rust MCP server providing Linux desktop automation through AT-SPI accessibility, desktop environment APIs, Wayland portals, and simulated keyboard/mouse input.

## 1. What the project actually does

Its key functionality includes:

| Capability | Implementation |
|---|---|
| Screenshot capture | GNOME screenshot APIs, portals, X11 fallback |
| Mouse clicking | Accessibility actions or coordinates |
| Keyboard typing | Wayland input backends / X11 |
| Drag and drop | Desktop coordinate controls |
| Scrolling | Accessibility and pointer backends |
| Window discovery | GNOME, KDE, Hyprland, niri, i3, COSMIC, X11 |
| Window focus | Identify and activate applications |
| Window positioning | Move and resize where supported |
| Accessibility inspection | AT-SPI application element trees |
| Semantic UI operations | Find controls by role, name, text, or state |
| Shell execution | Optional, disabled by default |
| MCP integration | Native Rust MCP server |
| Diagnostics | `doctor` readiness report |

One particularly useful feature is that the agent can inspect the accessibility tree and interact with named UI elements instead of relying exclusively on screenshot coordinates.

For example, an agent could identify a **Save** button in a desktop application and invoke its accessibility action, rather than guessing where the button appears on-screen.

## 2. How it fits ContextBridge

ContextBridge and computer-use-linux solve different problems.

| Capability | ContextBridge target | computer-use-linux |
|---|---|---|
| Filesystem operations | Yes | Through optional shell |
| Terminal execution | Yes | Optional limited execution |
| Remote device management | Yes | No standalone relay |
| Windows support | Yes | No |
| Graphical desktop control | Planned | Yes |
| Accessibility automation | Planned | Yes |
| Bidirectional synchronization | Yes | No |
| Local MCP server | Yes | Yes |
| Authentication and authorization | Required | Relies primarily on its MCP host and local access boundary |

The main benefit is avoiding the need to independently implement Linux graphical automation.

Instead of writing your own mouse, keyboard, screenshot, and accessibility control backends, ContextBridge could incorporate this Rust implementation.

## 3. Code quality assessment

I inspected the repository's source layout, MCP server implementation, process execution code, dependency manifest, and security documentation.

### Modular architecture

Separate modules handle accessibility, screenshots, window targeting, keyboard input, command execution, and diagnostics. This makes selective reuse practical.

### Process supervision

The command runner implements timeouts, output limits, process-group termination, and cleanup on cancellation. These are worthwhile patterns for ContextBridge's terminal engine.

### Careful MCP tool registration

Shell execution is removed from the advertised tools unless `COMPUTER_USE_LINUX_ENABLE_SHELL=1`. Tool annotations distinguish observation from state-changing operations.

### Resource boundaries

Screenshot dimensions and payload sizes are capped. Terminal output is bounded, and several potentially slow operations have deadlines.

*These findings indicate a thoughtfully engineered implementation, although I have not executed the project or completed a full security audit.*

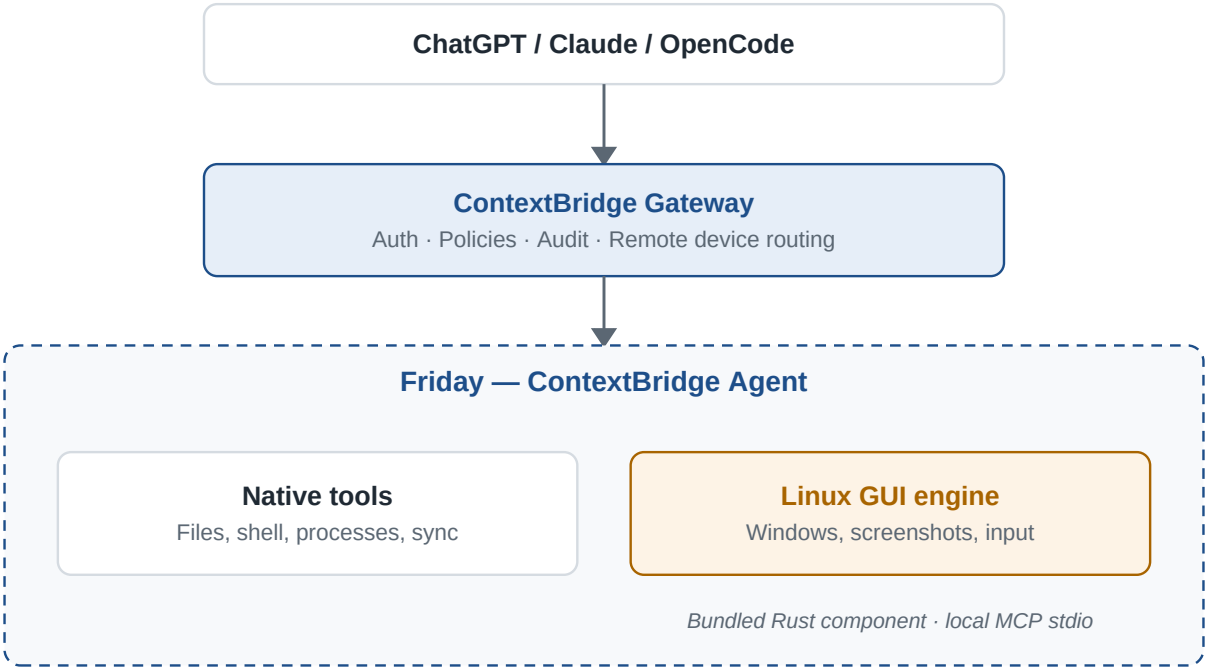
## 4. Risks and limitations

| Severity | Finding | Implication |
|---|---|---|
| **High** | No independent remote authorization layer | ContextBridge must authorize every computer-control request before forwarding it |
| **High** | Shell access is effectively host-user access | Timeouts and output limits are not a security sandbox |
| **Medium** | GUI automation requires an active desktop session | A system service running without access to the graphical user session may fail |
| **Medium** | GNOME-specific functionality varies by compositor | Friday's actual desktop environment needs testing |
| **Medium** | No native Windows implementation | Thanatos needs a separate backend |
| **Low** | Rust and Python integration | Requires process supervision, protocol integration, and packaging coordination |

> **Important for Friday**
>
> The project reports manual validation on Ubuntu GNOME Wayland, with additional compositor-specific testing.
>
> Friday may be running Cinnamon or XFCE components, based on its installed desktop applications. That does not establish which desktop session is active.
>
> If Friday uses X11, the project has generic X11/EWMH support. If it uses GNOME Wayland, the GNOME backend is more mature.
>
> I would run its `doctor` command before deciding which components to integrate.

## 5. The architecture I recommend

Rather than porting all this Rust code into Python, make it a locally bundled subsystem of ContextBridge.




This is still an independent architecture because ContextBridge would own the agent, distribution, authentication, tool authorization, and remote connectivity. The Rust engine would be a locally bundled open-source dependency, not an external hosted service.

The project's MIT license permits reuse and modification, provided required copyright and license notices are preserved.

If the goal is independence from all external maintained software, you could vendor a reviewed version into ContextBridge and maintain it yourself.

## 6. What I would reuse

My preference is selective integration, not replacing the ContextBridge code already being developed.

| Source component | Proposed ContextBridge use | Priority |
|---|---|---|
| `atspi_tree.rs` | Semantic UI discovery and actions | **High** |
| `windowing/` | Window identification and focus | **High** |
| `screenshot.rs` | Screen capture | **High** |
| `remote_desktop.rs` | Wayland input support | **High** |
| `ydotool.rs` | Input fallback | **High** |
| `diagnostics.rs` | Desktop readiness checks | **High** |
| `command_runner.rs` | Reference for process safety | **Medium** |
| `indicator.rs` | Visual indication of agent activity | **Medium** |
| `server.rs` | MCP integration reference | **Medium** |

> **Essential security change**
>
> The ContextBridge agent must authorize GUI actions locally as well as at the gateway. An approved screenshot request should not imply permission to type commands into a terminal or approve desktop security prompts.

## 7. Final assessment

**Assessment for ContextBridge**

| | |
|---|---|
| Linux GUI functionality<br>**Strong** | Remote infrastructure<br>**Not included** |
| Implementation independence<br>**Good** | Windows compatibility<br>**None** |

I would adopt computer-use-linux as the Linux graphical control engine, while continuing to build ContextBridge's independent filesystem, terminal, synchronization, authorization, and remote-agent capabilities.

That gets you closer to something broader than Desktop Commander:

- Desktop Commander-style filesystem and terminal control.
- Graphical desktop control through screenshots and accessibility.
- Native remote agents for Friday and Thanatos.

- Full bidirectional file and directory synchronization.
- Self-hosted remote access without Desktop Commander or Authentik.

The next practical step is a read-only compatibility test on Friday: identify its desktop session, inspect the repository's required system components, and run the diagnostic tool without enabling input or modifying desktop settings.

> **Bottom line**
>
> This repository is worth integrating into ContextBridge. It is a strong potential building block for the Linux desktop-control layer, but not a replacement for ContextBridge's remote gateway or security model.